

RDD Quick Revision

February 22, 2026

```
[1]: from pyspark.sql import SparkSession

spark=SparkSession.builder.appName("revision").getOrCreate()

sc=spark.sparkContext
```

```
[ ]: # partitioning
rdd.getNumPartitions()

# set partitions
sc.parallelize(data,4)

# repartition
rdd.repartition(4)
rdd.coalesce(1)
```

```
[8]: # creation from collcection
l=[2,3,4,5]
rdd1=sc.parallelize(l)

# from external file
rdd=sc.textFile("prod.csv")
rdd.collect()
# rdd.take(10)
# header
header= rdd.first()
```

```
[7]: # Transformations --> map,filter,flatMap,distinct,union,join,reduceByKey
# actions(execute the dag)-->collect,count,first,take,reduce,saveAsTextFile
```

```
[9]: # remove header
no_h=rdd.filter(lambda x:x!=header)
```

```
[11]: # Map
# one input --> one output
r1=sc.parallelize(["This is random String"])
r1.map(lambda x:x.split(" ")).collect()
```

```
[11]: [['This', 'is', 'random', 'String']]
```

```
[13]: # FlatMap() --> one input --> multiple outputs(flattened)
r1.flatMap(lambda x:x.split(" ")).collect()
```

```
[13]: ['This', 'is', 'random', 'String']
```

```
[14]: # filter --> filter based on condition
```

```
[17]: # Key - Value RDD
# (key,value)
rdd = sc.parallelize(["a 1", "b 2", "a 3"])
pairs = rdd.map(lambda x: (x.split()[0], int(x.split()[1])))
pairs.collect()
```

```
[17]: [('a', 1), ('b', 2), ('a', 3)]
```

```
[18]: # reduceByKey()
# efficient aggregation
pairs.reduceByKey(lambda x,y:x+y).collect()
```

```
[18]: [('a', 4), ('b', 2)]
```

```
[19]: # groupByKey --> slower
pairs.groupByKey().mapValues(sum).collect()
```

```
[19]: [('a', 4), ('b', 2)]
```

```
[ ]: # word count example
r1=sc.textFile("sample.txt")
wordcount=(
    r1.flatMap(lambda x:x.split(" "))
    .map(lambda x:(x,1)).reduceByKey(lambda x,y:x+y)
)
```

In RDD, “group by column” = Convert column into KEY → then groupByKey/reduceByKey.

```
[ ]: # Example , group by city
pair_dd=rdd.filter(lambda x:x!=header).map(lambda x:x.split(",")).map(lambda x:
    ↪(x[2],[3]))
#(city,name)
# now
pair_rdd.groupByKey().collect()

# or
pair_rdd.mapValues(lambda x:list(x)).collect()
```

```
[ ]: # distinct
rdd.distinct() # removes duplicates

# union
rdd1.union(rdd2)

# intersection
# common elements
```

```
[ ]: # sorting
# sortByKey()
pairs.sortByKey().collect()

# sortBy value
rdd.sortBy(lambda x:x[1],ascending=False) # descending
```

```
[ ]: # Join
rdd1 = sc.parallelize([(1, "Tarun"), (2, "Ali")])
rdd2 = sc.parallelize([(1, "Premium"), (2, "Regular")])

rdd1.join(rdd2).collect()
```

```
[ ]: # Narrow Transformations (no shuffle) --> map,filter,flatMap
# wide (shuffle happens)--> reduceByKey,groupByKey,join,distinct
```

```
[ ]: # groupByKey() --> when we need all values of a key together, not just sum/
    ↪ count.
# it gives list of values

# ex: find all products bought by each customer
orders = sc.parallelize([("C1", "P1"), ("C1", "P2"), ("C2", "P3"), ("C1", "P4")])

result = orders.groupByKey().mapValues(list)

result.collect()
```

```
[ ]: [( 'C2', ['P3']), ('C1', ['P1', 'P2', 'P4'])]
```

```
[3]: # reduceByKey() --> when we need to aggregate values(sum,count,max,min,avg,
    ↪ logic)
# its more efficient than groupBy

# ex: find total quantity sold per product
orders = sc.parallelize([("P1", 2), ("P1", 3), ("P2", 1), ("P1", 4)])

result = orders.reduceByKey(lambda x, y: x + y)
```

```
result.collect()
```

```
[3]: [('P1', 9), ('P2', 1)]
```

If you want	Use
Sum	reduceByKey
Count	reduceByKey
Max	reduceByKey
List of values	groupByKey

```
[5]: # mapValues() --> when we want to modify only the values ,not the key
rdd = sc.parallelize([("P1", 10), ("P2", 20)])
result = rdd.mapValues(lambda x: x * 2)
result.collect()
```

```
[5]: [('P1', 20), ('P2', 40)]
```

```
[ ]: # flatMap() --> one input --> multiple outputs(flattens the result)
# ex: count words in sentences
```

Function	Output Type	Use Case
groupByKey	(k, list)	Need all values
reduceByKey	(k, single value)	Aggregation
mapValues	modifies value only	Clean value transformation
flatMap	multiple outputs per input	Word split, explode

```
[ ]: # proper template
rdd = sc.textFile("file.txt")

header = rdd.first()

cleaned = (
    rdd.filter(lambda x: x != header)
    .map(lambda x: x.split(","))
    .filter(lambda x: len(x) == expected_columns)
)
```

```
[ ]: # save into a file
rdd.saveAsTextFile("test.csv") # saves into a folder(in that multiple
    ↪ parts, (partitions))

# to save into a single one, we use coalesce(1) --> means to force into 1
    ↪ partition
rdd.coalesce(1).saveAsTextFile("test.csv")
```

Full Example

```
[ ]: # Full qsn
# For each city, calculate:
# Total sales
# Total number of orders
# Average order value
# Only keep cities where total sales > 700

rdd=sc.textFile("orders.txt")
h=rdd.first()

cleaned=rdd.filter(lambda x:x!=h).map(lambda x:x.split(","))
cleaned.collect()

# (key,value)
# key =city, value=(amount,1)
pair=cleaned.map(lambda x:(x[2],(float(x[4]),1))) # this is for better shuffle,
↳we can do it separately also(but this takes multiple shuffles)

# aggregations
# sum of amount(total sales)
# total number of orders
aggregated=pair.reduceByKey(lambda x,y:(x[0]+y[0],x[1]+y[1])) # (total
↳sales,total orders)

# avg
final=aggregated.mapValues(lambda x:(x[0],x[1],x[0]/x[1]))

# filter total sales>700
res=final.filter(lambda x:x[1][0]>700)
# sort by total sales descending
res1=res.sortBy(lambda x:x[1][0],ascending=False)

#print
for city,values in res1.collect():
    print(f"{city},{values[0]},3,{values[1]},3,{values[2]}")
```

Houston,1000.0,3,333.3333333333333

```
[ ]: # max order value per city
# min order value per city

pair=cleaned.map(lambda x:(x[2],float(x[4])))

max_order=pair.reduceByKey(lambda x,y:x if x>y else y) # or lambda x,y:max(x,y)

min_order = pair.reduceByKey(lambda x, y: x if x < y else y)
```

BASIC PATTERNS

Question Type	What To Use
Count total rows	<code>rdd.count()</code>
Distinct values	<code>rdd.distinct()</code>
Filter condition	<code>rdd.filter()</code>
Sort ascending	<code>sortBy(lambda x: x[1])</code>
Sort descending	<code>sortBy(lambda x: x[1], ascending=False)</code>
Top N	<code>sortBy(...).take(N)</code>

AGGREGATION PATTERNS

Question Type	What To Use
Count per key	<code>(key,1) + reduceByKey(lambda x,y: x+y)</code>
Sum per key	<code>(key,value) + reduceByKey(lambda x,y: x+y)</code>
Max per key	<code>reduceByKey(lambda x,y: x if x>y else y)</code>
Min per key	<code>reduceByKey(lambda x,y: x if x<y else y)</code>
Average per key	<code>(key,(value,1)) + reduceByKey → divide sum/count</code>
Multiple aggregations	<code>(key,(count,sum)) + reduceByKey</code>
Collect all values	<code>groupByKey().mapValues(list)</code>

JOIN PATTERNS

Question Type	What To Use
Inner Join	<code>rdd1.join(rdd2)</code>
Left Join	<code>rdd1.leftOuterJoin(rdd2)</code>
Right Join	<code>rdd1.rightOuterJoin(rdd2)</code>
Anti Join	<code>leftOuterJoin + filter(None)</code>
3 Table Join	<code>Join1 → map → Join2</code>

TRANSFORMATION CHEATSHEET

Function	Use When
<code>map</code>	One input → One output
<code>flatMap</code>	One input → Many outputs
<code>filter</code>	Condition filtering
<code>mapValues</code>	Change only value
<code>groupByKey</code>	Need list of values
<code>reduceByKey</code>	Need aggregation
<code>join</code>	Combine datasets

FLATMAP

Problem	Use
Word count	<code>flatMap(split)</code>
Explode column	<code>flatMap()</code>
Split comma values	<code>flatMap(lambda x: x.split(","))</code>

COMMON EXAM STRUCTURES

Count customers per state

```
cust.map(lambda x: (x[4],1)) \
    .reduceByKey(lambda x,y: x+y)
```

Total revenue per product

```
orders.map(lambda x: (x[2], int(x[3]))) \
    .reduceByKey(lambda x,y: x+y)
```

Average age per type

```
cust.map(lambda x: (x[5], (int(x[6]),1))) \
    .reduceByKey(lambda x,y: (x[0]+y[0], x[1]+y[1])) \
    .mapValues(lambda x: x[0]/x[1])
```

Top spender per state

```
state_level.reduceByKey(
    lambda x,y: x if x[1] > y[1] else y
)
```

Customers with > 2 orders

```
orders.map(lambda x: (x[1],1)) \
    .reduceByKey(lambda x,y:x+y) \
    .filter(lambda x:x[1]>2)
```

ANTI JOIN PATTERN

```
left.leftOuterJoin(right) .filter(lambda x: x[1][1] is None)
```

MULTI AGGREGATION TEMPLATE

```
rdd.map(lambda x: (key, (1, value))).reduceByKey(lambda x,y: (x[0]+y[0], x[1]+y[1]))
```

EXAM SHORTCUT

Words in Question	Immediately Think
“Total”	<code>reduceByKey</code>
“Count”	<code>(key,1)</code>

Words in Question	Immediately Think
“Average”	(value,1) pattern
“Top”	sortBy descending
“Max per”	reduceByKey compare
“Never”	leftOuterJoin + filter
“Distinct”	distinct()
“More than”	filter after aggregation

When question appears:

- 1 Identify KEY
- 2 Identify VALUE
- 3 Decide → reduceByKey or groupByKey
- 4 If multiple tables → join first
- 5 Then aggregate